

Sprawozdanie Projekt PPD

Rozmieszczenie Stacji Bazowych GSM

Jakub Wieśniak 276187

Cel:

Głównym celem projektu jest zastosowanie modelowania i rozwiązywania problemów optymalizacyjnych do rozwiązania problemu rozmieszczenia stacji bazowych GSM na terenie o niejednostajnej gęstości zaludnienia. Rozwiążemy ten problem przy pomocy narzędzia CPLEX oraz metaheurystyki Particle Swarm Optimization PSO w Python.

Opis problemu:

Musimy dostarczyć mieszkańcom żyjącym na terenie o niejednostajnej gęstości zaludnienia dostęp do usług telekomunikacyjnych realizowanych poprzez stacje bazowe posiadające ograniczony zasięg działania. Posiadamy ograniczony budżet by dokonać tego zadania dlatego podczas stawiania wież musimy zadbać o to by swoim działaniem obejmowały obszar który obsłuży jak największą liczbę przyszłych użytkowników.

Sformułowanie problemu optymalizacyjnego:

Parametry:

- **L** - Lista dostępnych lokalizacji dla stacji bazowych
- **K** - Lista ośrodków potrzebujących usług telekomunikacyjnych
- **P** - Ilość stacji, które możemy wybudować
- **station_range[1..P]** - Wektor zasięgów dostępnych stacji
- **station_location[1..L][1..2]** - Wektor dostępnych lokalizacji
- **settlement_location[1..K][1..2]** - Wektor ośrodków potrzebujących usług telekomunikacyjnych

Zmienne decyzyjne:

- **x[1..P][1..L]** - Zmienna decyzyjna odpowiadająca za wybór lokalizacji umieszczenia stacji bazowej
- **covered[1..K]** - Zmienna pomocnicza określająca, które ośrodki są już obsłużone
- **uncovered** - Całkowita liczba nieobsłużonych ośrodków

Funkcja celu:

Celem jest minimalizacja liczby nieobsłużonych ośrodków którym nie będziemy mogli zapewnić usług telekomunikacyjnych.

Ograniczenia:

- Każda stacja może być postawiona tylko w jednej lokalizacji
- Każda lokalizacja może posiadać tylko jedną stację
- Każda stacja może obsługiwać tylko ośrodki będące w zasięgu jej działania

Metaheurystyka PSO:

Zasada działania algorytmu PSO:

Ideą algorytmu PSO jest iteracyjne przeszukiwanie przestrzeni rozwiązań problemu przy pomocy roju cząstek. Każda z cząstek posiada swoją pozycję w przestrzeni rozwiązań, prędkość oraz kierunek w jakim się porusza. Ponadto zapamiętywane jest najlepsze rozwiązanie znalezione do tej pory przez każdą z cząstek (rozwiązanie lokalne), a także najlepsze rozwiązanie z całego roju (rozwiązanie globalne). Prędkość ruchu poszczególnych cząstek zależy od położenia najlepszego globalnego i lokalnego rozwiązania oraz od prędkości w poprzednich krokach. Poniżej przedstawiony jest wzór pozwalający na obliczenie prędkości danej cząstki.

$$v \leftarrow \omega v + \phi_1 r_1 (l - x) + \phi_2 r_2 (g - x)$$

Gdzie:

- v - prędkość cząstki
- ω - współczynnik bezwładności, określa wpływ prędkości w poprzednim kroku
- ϕ - współczynnik dążenia do najlepszego lokalnego rozwiązania
- ϕ_g - współczynnik dążenia do najlepszego globalnego rozwiązania
- l - położenie najlepszego lokalnego rozwiązania
- g - położenie najlepszego globalnego rozwiązania
- x - położenie cząstki
- r_1, r_2 - losowe wartości z przedziału $<0,1>$

Powyższy wzór pozwala na aktualizację prędkości wszystkich cząstek na podstawie uzyskanej do tej pory wiedzy.

Schemat działania algorytmu PSO:

- Dla każdej cząstki ze zbioru:
 - Wylosuj pozycję początkową z przestrzeni rozwiązań
 - Zapisz aktualną pozycję cząstki jako najlepsze lokalne rozwiązanie
 - Jeśli rozwiązanie to jest lepsze od najlepszego rozwiązania globalnego, to zapisz je jako najlepsze
 - Wylosuj prędkość początkową
- Dopóki nie zostanie spełniony warunek stopu (np. minie określona liczba iteracji):
 - Dla każdej cząstki ze zbioru:
 - Wybierz losowe wartości parametrów r_1 i r_2
 - Zaktualizuj prędkość cząstki wg powyższego wzoru
 - Zaktualizuj położenie cząstki w przestrzeni
 - Jeśli aktualne rozwiązanie jest lepsze od najlepszego rozwiązania lokalnego:
 - Zapisz aktualne rozwiązanie jako najlepsze lokalnie
 - Jeśli aktualne rozwiązanie jest lepsze od najlepszego rozwiązania globalnego:

- Zapisz aktualne rozwiązanie jako najlepsze globalnie

Implementacja PSO w Python dla naszego problemu:

Reprezentacja cząstki (rozwiązania):

Funkcja: def initialize_particle():

Rola: W tej funkcji inicjalizujemy cząstkę, czyli jedno z potencjalnych rozwiązań problemu. Cząstka jest listą długości P (liczba stacji), gdzie każda wartość to indeks wybranej lokalizacji dla danej stacji.

Funkcja celu (fitness):

Funkcja: def evaluate_fitness(particle):

Rola: Funkcja celu ocenia, jak dobre jest dane rozwiązanie. Sprawdza, ile ośrodków jest obsłużonych przez stacje znajdujące się w wybranych lokalizacjach. Liczba nieobsłużonych ośrodków jest zwracana jako wartość funkcji celu.

Inicjalizacja cząstek:

Kod:

```
num_particles = 50
particles = [initialize_particle() for _ in range(num_particles)]
particle_fitness = [evaluate_fitness(particle) for particle in particles]
global_best_particle = min(particles, key=evaluate_fitness)
global_best_fitness = evaluate_fitness(global_best_particle)
```

Rola:

Inicjalizujemy populację cząstek oraz obliczamy ich wartości funkcji celu. Wybieramy również najlepsze rozwiązanie spośród początkowych cząstek jako globalne najlepsze rozwiązanie.

Parametry PSO:

Kod:

```
w = 0.9 # Współczynnik bezwładności
c1 = 2.0 # Współczynnik kognitywny
c2 = 2.0 # Współczynnik społeczny
max_iterations = 100
```

Rola:

Ustalamy parametry dla algorytmu optymalizacji roju cząstek (PSO), które wpływają na ruch cząstek w przestrzeni rozwiązań.

Główna pętla PSO:

Kod:

```
for iteration in range(max_iterations):
    for i in range(num_particles): ...
```

Rola:

W tej pętli algorytm PSO aktualizuje położenia cząstek, biorąc pod uwagę ich osobiste najlepsze rozwiązania oraz najlepsze globalne rozwiązanie. Aktualizowane są również wartości funkcji celu.

Rozwiązania:

Dane do rozwiązania:

// Lista dostępnych lokalizacji dla stacji bazowych

int L = 12;

// Lista ośrodków potrzebujących usług telekomunikacyjnych

int K = 14;

// Ilość stacji, które możemy wybudować

int P = 3;

// Wektor zasięgów dostępnych stacji

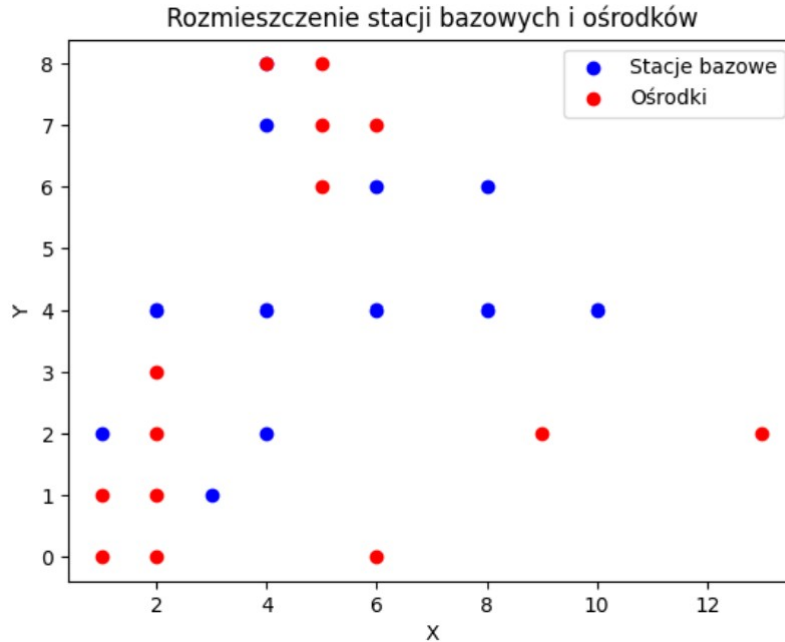
float station_range[1..P] = [1, 2, 3];

// Wektor dostępnych lokalizacji

float station_location[1..L][1..2] = [[4, 2], [2, 4], [6, 4], [6, 6], [8, 4], [4, 8], [10, 4], [8, 6], [4, 4], [3, 1], [1, 2], [4, 7]];

// Wektor ośrodków potrzebujących usług telekomunikacyjnych

float settlement_location[1..K][1..2] = [[2, 0], [2, 1], [2, 2], [2, 3], [1, 0], [1, 1], [6, 0], [9, 2], [13, 2], [5, 6], [5, 7], [5, 8], [4, 8], [6, 7]];



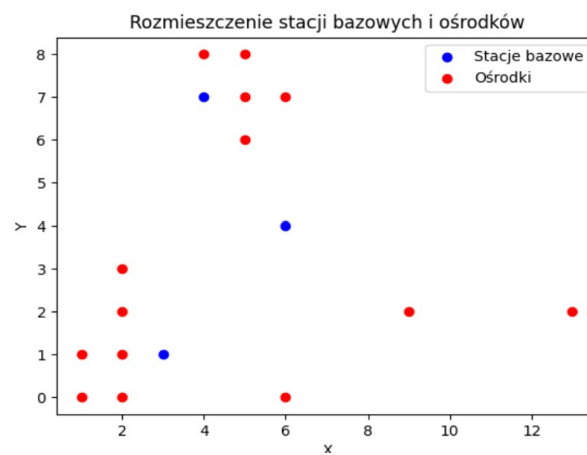
Rysunek 1: Wizualizacja punktów

Kod CPLEX:

```
uncovered = 3;  
x = [[0 0 1 0 0 0 0 0 0 0 0]  
      [0 0 0 0 0 0 0 0 0 0 1]  
      [0 0 0 0 0 0 0 0 0 1 0]];  
covered = [1 1 1 1 1 1 0 0 0 1 1 1 1];
```

Rysunek 2: Wynik CPLEX

Zmienna X odpowiada wybraniem punktów: $[[6,4], [4,7], [3,1]]$



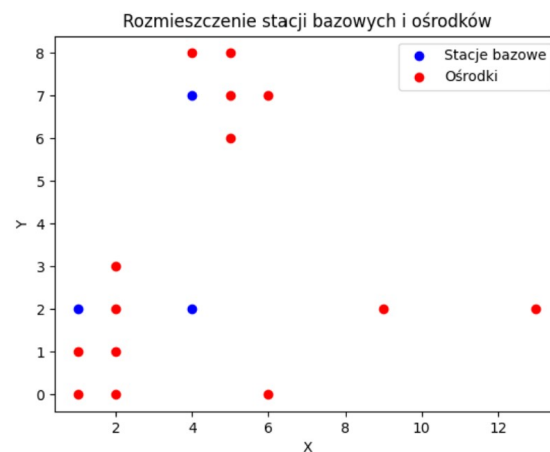
Rysunek 3: Wizualizacja

Kod Python:

```
Najlepsze rozmieszczenie stacji bazowych:  
Stacja 1: [4, 2]  
Stacja 2: [4, 7]  
Stacja 3: [1, 2]  
Liczba nieobsłużonych ośrodków: 3
```

Rysunek 4: Wyniki PSO

Zmienna X odpowiada wybraniem punktów: $[4,2], [4,7], [1,2]$



Rysunek 5: Wizualizacja

Podsumowanie:

Niniejszy projekt ilustruje praktyczne zastosowanie technik modelowania i optymalizacji w rozwiązywaniu złożonych problemów rzeczywistych takich jak wybór lokalizacji. Istotnym elementem analizy było uwzględnienie paru ograniczeń występujących w zadaniu. Porównując strategie wypracowane za pomocą Pythona i narzędzia optymalizacyjnego CPLEX, możemy zauważyć różnicę między obiema metodami.